TypeScript Interview Questions — Quick Revision (Most Important Only)

> Short version. Only the questions asked most often. For full depth, see `02-typescript-interview-questions.md`.

Table of Contents

---

1. What is TypeScript

Answer

TypeScript is JavaScript with types added on top. It compiles down to normal JS. The main benefit: it catches mistakes while you write code, not when the app crashes at runtime. Also gives better autocomplete in large codebases.

Example

```
function add(a: number, b: number): number
{
  return a + b;
}
add("5", 10); // Compile error, caught
immediately
```

Real-world Example

Stops you from typing `user.naem` instead of `user.name` — caught instantly in the editor instead of after running the app.

Follow-up Questions

- Does TS code run directly, or get compiled first?

- Can you use plain `.js` files inside a TS project?

---

2. Interface vs Type Alias

Answer

Both describe object shapes and work the same in most cases. `interface` can be extended and reopened later to add more properties. `type` cannot be reopened, but is more flexible — can describe unions, tuples, primitives too. Rule of thumb: `interface` for object/API models, `type` for unions.

Example

```
interface User { id: number; name: string;
}
type Status = "loading" | "success" |
"error"; // type can do unions
```

Real-world Example

`interface UserResponse` for API models, `type ScreenState = "loading" | "success" | "error"` for screen states.

Follow-up Questions

- What is declaration merging?
- Which one would you use for Redux state shape, and why?

---

3. Union vs Intersection Types

Answer

Union (`|`) means a value can be one of several types ("OR"). Intersection (`&`) means a value must satisfy all combined types at once ("AND").

Example

```
type ID = string | number;          //
union
type Person = { name: string } & { age:
number }; // intersection
```

Real-world Example

Union for API status (`"idle" | "loading" | "error"`). Intersection to merge a base type with extra fields (`User & { token: string }`).

Follow-up Questions

- How is a union different from an enum?

- How would you narrow a union type using a switch statement?

---

4. Generics

Answer

Generics let you write a function/interface/component that works with any type, while still staying type-safe — instead of duplicating code or using `any` and losing safety. `T` is a placeholder type filled in when used.

Example

```
function getFirst<T>(arr: T[]): T {
  return arr[0];
}
interface ApiResponse<T> { data: T;
success: boolean; }
```

Real-world Example

`function fetchData<T>(url: string): Promise<T>` reused for users, products, orders with correct typing each time.

Follow-up Questions

- Why use generics instead of `any`?
- Can you write a generic React component?

---

5. any vs unknown vs never vs void

Answer

`any` turns off type checking — avoid it. `unknown` can hold anything but you must check its type before using it — safer than `any`. `void` means a function returns nothing useful. `never` means a function never successfully returns (always throws, or infinite loop).

Example

```
let v: unknown = 5;
if (typeof v === "string") v.toUpperCase();
// must check first

function logMsg(msg: string): void {
console.log(msg); }
function fail(msg: string): never { throw
new Error(msg); }
```

Real-world Example

`unknown` is the correct type for `catch (error: unknown)` in modern TS, since the error's shape isn't known ahead of time.

Follow-up Questions

- Why is `unknown` safer than `any`?
- When would a function's return type be `never`?

---

6. Partial, Required, Pick, Omit, Record

Answer

Built-in utility types that transform an existing type. `Partial<T>` makes all fields optional. `Required<T>` makes all fields mandatory. `Pick<T, K>` keeps only chosen fields. `Omit<T, K>` removes chosen fields. `Record<K, V>` builds an object type with keys of type `K` and values of type `V`.

Example

```
interface User { id: number; name: string;
email: string; }
type UserUpdate = Partial<User>;
// edit form
type UserPreview = Pick<User, "id"|"name">;
```

```
// list item
type NoEmail = Omit<User, "email">;
```

Real-world Example

`Partial<User>` for an "Edit Profile" form. `Pick` for a lightweight FlatList item type.

Follow-up Questions

- How would `Omit` help build a "create user" type that excludes `id`?
- Can utility types be combined?

---

7. Type Guards vs Type Assertions

Answer

A type guard actually checks the type at runtime (`typeof`, `instanceof`, `in`) — safe. A type assertion (`as`) just tells TS "trust me," without checking anything — risky if you're wrong.

Example

```
function print(v: string | number) {
  if (typeof v === "string")
v.toUpperCase(); // type guard
```

```
}
const data = response.data as User; // type
assertion
```

Real-world Example

Type guards to safely narrow an API response that could be success or error shape. Assertions used (carefully) for JSON you're confident about.

Follow-up Questions

- Why is a type guard safer than an assertion?
- What is a custom type guard (`value is Type`)?

---

8. keyof and typeof

Answer

`keyof` takes an object type and gives a union of its property names. `typeof` (in the type world) takes an existing variable's type and reuses it as a TS type, so you don't redefine the shape manually.

Example

```
interface User { id: number; name: string;
}
```

```
type UserKeys = keyof User; // "id" |
"name"

const defaultUser = { id: 1, name: "Asha"
};
type DefaultUserType = typeof defaultUser;
```

Real-world Example

`keyof` used in a generic `getProperty(user, key)` helper so only valid keys are accepted with autocomplete.

Follow-up Questions

- How is `typeof` in TS different from `typeof` in JS?
- Can `keyof` be combined with generics?

---

9. Enums

Answer

A set of named constant values, more readable than raw strings/numbers scattered everywhere. String enums (you assign each value) are usually preferred for readability in logs.

Example

```
enum OrderStatus {
  Pending = "PENDING",
  Shipped = "SHIPPED",
}
```

Real-world Example

Used for order status, user roles (`Admin` , `User` , `Guest`).

Follow-up Questions

- Why might a team prefer a string union over an enum?
- Numeric vs string enum — difference?

---

10. Tuples

Answer

A fixed-length array where each position has a known, specific type — unlike a normal array of one repeated type.

Example

```
type UserTuple = [number, string]; // id, name
```

```
const [count, setCount] = useState<number>
(0); // useState returns a tuple
```

Real-world Example

`useState` itself returns a tuple — that's why you destructure it in a fixed `[value, setter]` order.

Follow-up Questions

- Why does `useState` return a tuple instead of an object?
- Can tuple elements be optional?

---

11. Optional Properties & readonly

Answer

`?` after a property name makes it optional (may or may not be present). `readonly` means it can be set once (usually at creation) but never reassigned later.

Example

```
interface User {
  readonly id: number;
  nickname?: string;
}
```

Real-world Example

`readonly` for `id` / `createdAt` fields that should never change. Optional for fields an API might not always return.

Follow-up Questions

- Does `readonly` give real runtime protection, or only compile-time?
- How is `readonly` different from `Object.freeze()`?

---

12. extends vs implements

Answer

`extends` is for inheritance — a class/interface builds on another. `implements` is used on a class to force it to follow an interface's structure (must include all its properties/methods).

Example

```
interface Shape { area(): number; }
class Circle implements Shape {
  constructor(private radius: number) {}
  area() { return Math.PI * this.radius **
```

```
2; }
}
```

Real-world Example

`class AuthService implements IAuthService` guarantees `login`, `logout`, `refreshToken` exist — useful if you swap implementations later (e.g. for testing).

Follow-up Questions

- Can a class implement multiple interfaces?
- Can a class extend a class and implement an interface at the same time?

← Back to top